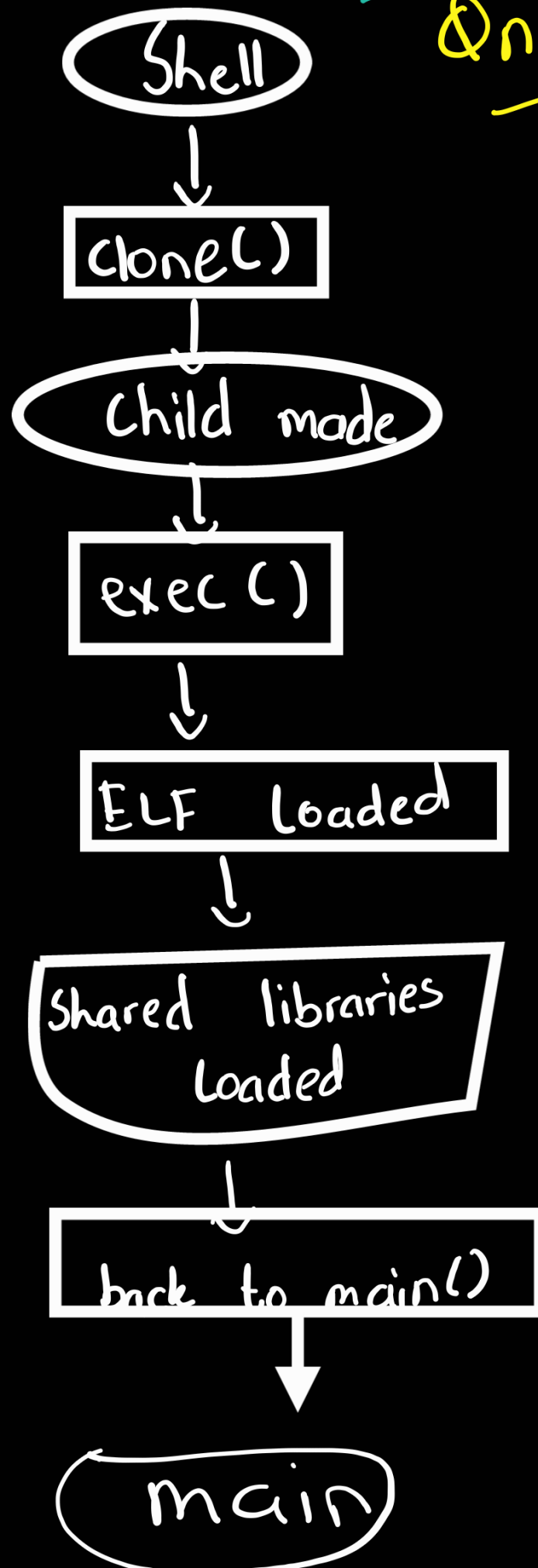


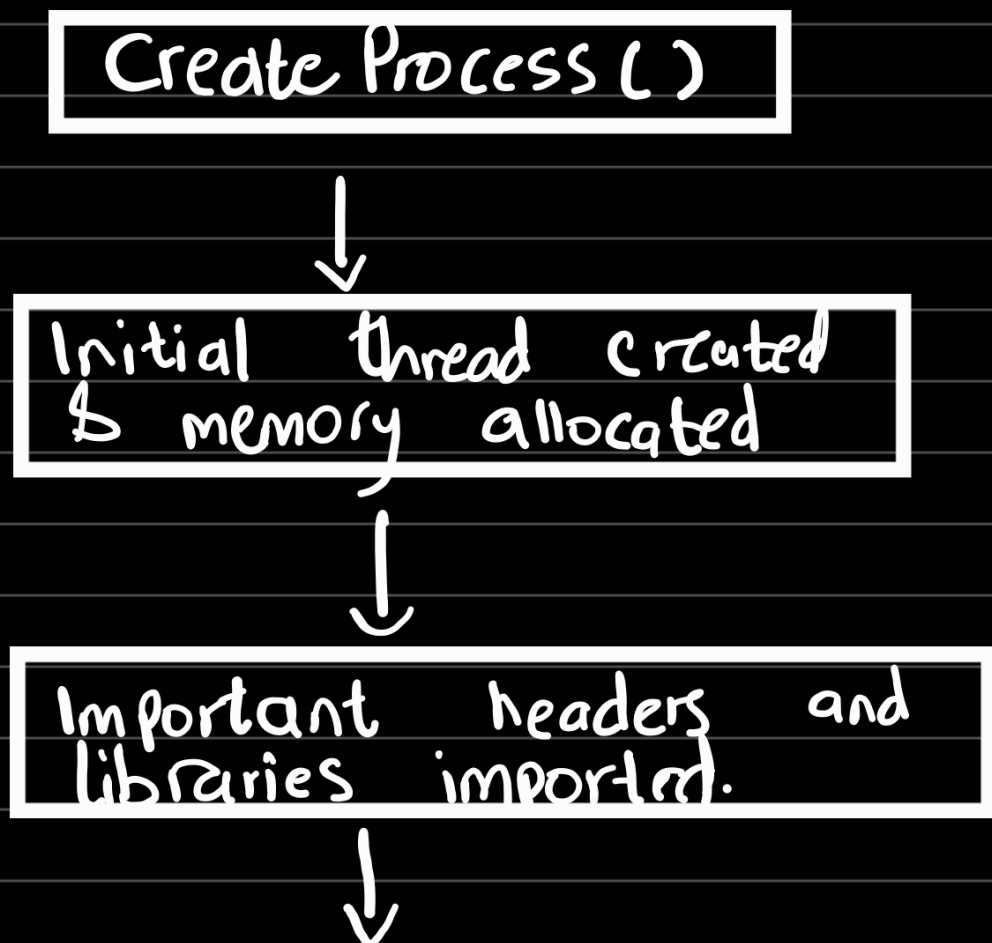
Program Starting in Linux

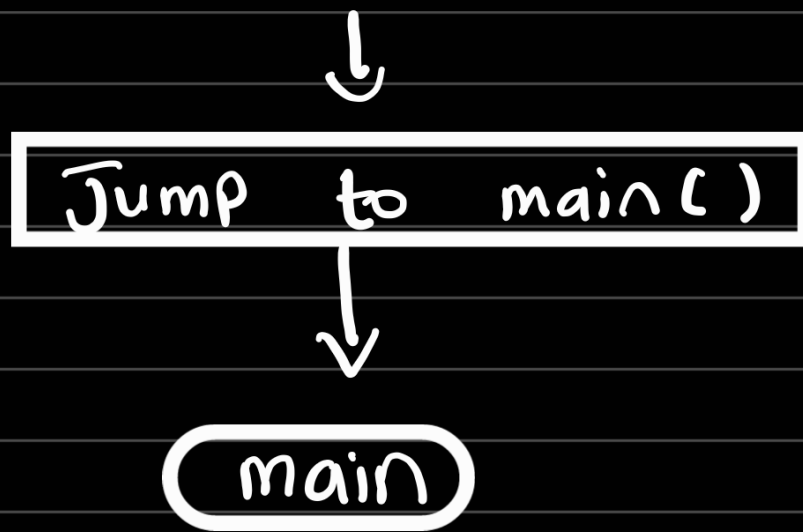
Qno.2.



Similar to xv6, Linux calls `clone()` to copy the whole parent to the child process. A little difference is resource allocation as `clone()` allocates sensible resources. The `execve()` call then brings a new program in the child process. ELF starts loading at this point. Next, required libraries are also loaded. The control is then transferred to the `main()` function of the newly loaded program.

WINDOWS





Create Process() call is done from the currently running program. This initializes a new process and its initial threads. address spaces are then initialised and memory is allocated. Base executable is loaded in these address spaces. Then, the PE loader imports the important libraries and dependency. The control is then transferred to the main function of the new process.

Q no. 3

```
int
main(void)
{
    static char buf[CMD_BUF_SIZE];
    int fd;

    // Ensure that three file descriptors are open.
    while((fd = open("console", O_RDWR)) >= 0){
        if(fd >= 3){
            close(fd);
            break;
        }
    }
}
```

→ buffer allocated of size in the variable.

→ fd = open opens 3 file descriptors i.e: stdin, stdout, stderr).

```
// Read and run input commands.
while(getcmd(buf, sizeof(buf)) >= 0){
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
        cmd++;
    if (*cmd == '\0') // blank command
        continue;

    addCommand(cmd); // Save to history

    if(cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ')
        cmd[strlen(cmd)-1] = 0; // chop \n
    if(chdir(cmd+3) < 0)
        fprintf(2, "cannot cd %s\n", cmd+3);
}
```

→ getcmd reads what's written on shell. Checks exist to avoid blanks.

→ Special case for cd

for directory navigation.

```
int
getcmd(char *buf, int nbuf)
{
    write(2, "$ ", 2);
    memset(buf, 0, nbuf);
    gets(buf, nbuf);
    if(buf[0] == 0) // EOF
        return -1;
    return 0;
}
```

→ Prints \$ on the shell.

→ reads command into buffer

```

if(cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' '){
    cmd[strlen(cmd)-1] = 0; // chop \n
    if(chdir(cmd+3) < 0)
        fprintf(2, "cannot cd %s\n", cmd+3);
} else {
    if(fork() == 0)
        runcmd(parsecmd(cmd));
    wait(0);
}
}
exit(0);
}

```

→ After all exceptions are checked for, runcmd is called. After making sure the

program is currently in a child and not the parent itself. Types of commands and their runnings are defined, parsed and validated in the runcmd function.

→ wait(0) waits for the child process to return 0 i.e.: complete its execution.

Link to Videos of Q1,4

<https://drive.google.com/drive/folders/17yBcrctPzegEA70ECbi3xvsSTD7DWGqY?usp=sharing>